

■ APERTURE

Photography CRM — Complete System Architecture & Specifications

Author: Antigravity AI Coding Assistant

Status: Production MVP Architecture

Target Stack: Python (FastAPI) & React (Vite)

Date: August 2026

1. System Overview

Aperture is a multi-tenant, production-grade SaaS platform designed for professional photographers and studios to manage client relationships, booking schedules, invoices, and photo proofing galleries. It implements a secure data-isolation architecture where every query is scoped to the authenticated studio (tenant) via JSON Web Tokens (JWT).

Key Pillars of the System:

- **Studio CRM Core:** Client directories, status pipes, and metadata history tracking.
- **Scheduling Calendar:** Multi-session booking with collision and buffer detection.
- **Invoices & Inbound Payments:** Quote conversions, base rate billing, tax calculation, and payment status updates.
- **Client Portal:** Unauthenticated portal dashboard linked to custom URLs for document signature and secure gallery access.
- **Gallery Proofing:** Premium grid with photo detail sliders, download features, and client favorites selection.

2. Database Schema & Architecture

The database layer runs on SQLite for development and transitions to PostgreSQL for staging. Multi-tenancy is enforced by adding a **studio_id** column to all tables. All queries filter on this ID after extracting it from the verified JWT payload.

Model Name	Primary Keys / Foreign Keys	Description & Scoping
Studio	id (PK)	Represents the studio/tenant name and license.
User	id (PK), studio_id (FK)	Photographers/Staff with encrypted passwords.
Client	id (PK), studio_id (FK)	Client directory records with styles preferences.
Booking	id (PK), client_id (FK), studio_id (FK)	Session scheduling logs and rate cards.
Invoice	id (PK), booking_id (FK), studio_id (FK)	Payment status tracker (Paid, Pending).
Gallery	id (PK), booking_id (FK), studio_id (FK)	Shared picture groups for client review.
Photo	id (PK), gallery_id (FK)	Image cards with original/edited URLs and AI tags.

3. Core API Endpoints

The backend is built with FastAPI. It segregates routes into standard auth, tenant-scoped paths, and public routes:

Authentication & Registration

- POST /api/auth/register: Registers a studio and its admin user.
- POST /api/auth/login: Verifies passwords and signs JWT tokens.

Dashboard & Management (JWT Protected)

- GET /api/dashboard/stats: Compiles statistics and upcoming shoots.
- GET /api/clients & POST /api/clients: Client directory CRUD.
- POST /api/bookings: Books a shoot. Warns if another session overlaps within 2 hours.

Public Routes (Token-free URL sharing)

- GET /api/public/clients/{id}/portal: Returns client portal dashboard data.
- GET /api/public/galleries/{id}: Fetches proofing photos.
- POST /api/public/photos/{id}/favorite: Saves client photo selection.

4. Running the Project Locally

Aperture includes a pre-configured root package.json which boots the entire system concurrently in a single terminal pane.

Setup and Start Command:

Open a terminal in the root folder C:\Users\udhaya durairaj\.gemini\antigravity\scratch\photography-crm and run:

```
npm run dev
```

This will start:

1. **FastAPI Server** on port 8000
2. **React/Vite App** on port 5173

5. Daily Git Workflow Guide

Follow this order of commands to save and share your daily progress safely:

1. Start of work (Get latest updates & branch):

```
git switch main git pull git switch -c features/my-new-updates
```

2. Stage and commit completed work:

```
git status git add . git commit -m "feat: added new features"
```

3. Push / Upload code to GitHub:

```
git push https://@github.com/udhayasmile14-png/photography-crm.git main
```